



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Evolución de un editor
Entidad-Relación con React y
mxGraph**



Presentado por Steven Paul Alba Alba
en Universidad de Burgos — 25 de junio
de 2026

Tutor: Jesús Manuel Maudes Raedo



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Jesús Manuel Maudes Raedo, profesor del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos.

Expone:

Que el alumno D. Steven Paul Alba Alba, con DNI 71755156D, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Evolución de un editor Entidad-Relación con React y mxGraph.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 25 de junio de 2026

Vº. Bº. del Tutor:

D. Jesús Manuel Maudes Raedo

Resumen

Este Trabajo Fin de Grado aborda la evolución de UBU E-R App, un editor web de diagramas Entidad-Relación desarrollado con React y mxGraph. El proyecto parte de una aplicación heredada y se centra en analizar su funcionamiento interno, estabilizar su núcleo funcional y ampliar de forma controlada los elementos del modelo Entidad-Relación soportados por la herramienta.

El trabajo se ha desarrollado con un enfoque incremental, prestando especial atención a la coherencia entre la representación gráfica del diagrama, el modelo interno mantenido por la aplicación, la validación semántica, la transformación al modelo relacional y la generación de código SQL. Entre las ampliaciones realizadas se incluyen entidades débiles, atributos compuestos y multivaluados, relaciones ternarias y un soporte inicial para jerarquías ISA. Además, se han incorporado mejoras de usabilidad, validación, importación, exportación y apoyo a la edición. El resultado debe entenderse como una evolución académica de la herramienta, orientada a mantener una traducción razonable entre el diagrama diseñado por el usuario y la salida relacional o SQL generada.

Descriptores

Modelo Entidad-Relación, diagramas E/R, React, mxGraph, transformación relacional, SQL, validación de modelos, aplicación web, jerarquías ISA

Abstract

This Bachelor's Thesis addresses the evolution of UBU E-R App, a web-based Entity-Relationship diagram editor developed with React and mxGraph. The project starts from an existing application and focuses on analysing its internal structure, stabilising its core behaviour and extending the support for several elements of the Entity-Relationship model.

The work has been carried out incrementally, paying special attention to the consistency between the visual representation of the diagram, the internal model maintained by the application, semantic validation, transformation into the relational model and SQL generation. The implemented extensions include support for weak entities, composite and multivalued attributes, ternary relationships and an initial support for ISA hierarchies. In addition, the project includes usability, validation, import, export and editing-support improvements. The resulting application should be understood as an academic evolution of the tool, aimed at maintaining a reasonable translation from user-designed Entity-Relationship diagrams to relational or SQL output.

Keywords

Entity-Relationship model, E/R diagrams, React, mxGraph, relational transformation, SQL, model validation, web application, ISA hierarchies

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
1.1. Contexto del proyecto	1
1.2. Motivación	2
1.3. Punto de partida	3
1.4. Problema abordado	3
1.5. Alcance del trabajo	3
1.6. Estructura de la memoria	4
1.7. Documentos adjuntos	5
2. Objetivos del proyecto	7
2.1. Objetivos generales	7
2.2. Objetivos específicos	7
3. Conceptos teóricos	9
3.1. Modelo Entidad-Relación	9
3.2. Elementos básicos del modelo	10
3.3. Elementos avanzados considerados	11
3.4. Transformación al modelo relacional	14
3.5. Generación de SQL	16
3.6. Alcance teórico aplicado en el proyecto	17

4. Técnicas y herramientas	19
4.1. Entorno de desarrollo web	19
4.2. Librerías principales del editor	19
4.3. Herramientas de validación y pruebas	20
4.4. Control de versiones, integración y despliegue	21
4.5. Documentación y redacción de la memoria	21
5. Aspectos relevantes del desarrollo del proyecto	23
5.1. Planteamiento y organización del desarrollo	23
5.2. Análisis y estabilización del sistema heredado	24
5.3. Evolución del modelo interno y persistencia	26
5.4. Ampliación de los elementos Entidad-Relación soportados	29
5.5. Validación, transformación relacional y generación de SQL	31
5.6. Refinamientos de usabilidad del editor	37
5.7. Pruebas y revisión manual	40
5.8. Dificultades y decisiones técnicas relevantes	42
6. Trabajos relacionados	45
6.1. Herramientas generales de diagramación	45
6.2. Herramientas orientadas al modelado de bases de datos	46
6.3. Comparativa frente a alternativas	47
7. Conclusiones y líneas de trabajo futuras	49
7.1. Conclusiones	49
7.2. Líneas de trabajo futuras	51
Bibliografía	53

Índice de figuras

1.1. Vista general del editor web con un diagrama Entidad-Relación cargado.	2
3.1. Ejemplo básico de diagrama Entidad-Relación con entidades, atributos, claves, una interrelación y cardinalidades.	10
3.2. Ejemplo de entidad débil asociada a una entidad propietaria mediante una relación identificadora.	12
3.3. Ejemplo de interrelación ternaria con tres entidades participantes y cardinalidades.	13
3.4. Ejemplo de jerarquía ISA con una entidad general y dos especializaciones.	14
3.5. Transformación relacional de una jerarquía ISA mediante tabla de generalización y tabla por especialización.	16
5.1. Relación entre el lienzo del editor, el modelo interno y las operaciones principales de la aplicación.	27
5.2. Diálogo de validación con errores agrupados del diagrama. . . .	33
5.3. Ejemplo de script SQL generado a partir de un diagrama validado. .	36
5.4. Acciones contextuales mostradas por el editor al seleccionar un elemento del diagrama.	38
5.5. Generación de una estructura básica de ejemplo en el editor. . .	40

Índice de tablas

4.1. Resumen de herramientas utilizadas en el proyecto.	22
6.1. Comparativa general entre herramientas relacionadas y la aplicación desarrollada.	48

1. Introducción

1.1. Contexto del proyecto

Cuando se diseña una base de datos, antes de crear tablas o escribir sentencias SQL, es necesario representar de forma conceptual la información del dominio y las relaciones existentes entre sus elementos. El modelo Entidad-Relación permite realizar esta representación mediante entidades, atributos e interrelaciones, facilitando el análisis previo al paso al modelo relacional.

Este Trabajo Fin de Grado se sitúa en ese contexto, pero no aborda el desarrollo de una herramienta desde cero. El proyecto se centra en la evolución de UBU E-R App, anteriormente denominada Draw E-R App, una aplicación web para la creación de diagramas Entidad-Relación desarrollada con React y mxGraph. La aplicación heredada ya ofrecía una base funcional, pero requería análisis, estabilización y ampliación para mejorar la coherencia entre el diagrama editado en el lienzo, el modelo interno, las validaciones, la transformación relacional y la generación de SQL.

La Figura 1.1 muestra una vista general del editor, donde se aprecia el lienzo de trabajo, el panel de acciones y la representación visual de un diagrama Entidad-Relación sencillo.

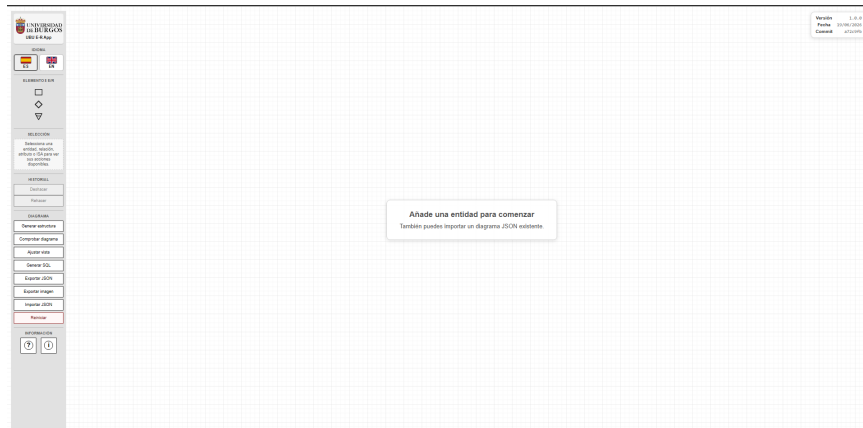


Figura 1.1: Vista general del editor web con un diagrama Entidad-Relación cargado.

1.2. Motivación

Aunque existen distintas herramientas para crear diagramas, muchas de ellas están pensadas como editores gráficos generales y no siempre encajan bien con el trabajo específico sobre diagramas Entidad-Relación. En estos diagramas no solo importa dibujar elementos, sino que también debe mantenerse una relación clara entre lo que se muestra en pantalla y el significado que tiene dentro del modelo.

La motivación de este trabajo no fue sustituir a esas herramientas generales de diagramación, sino continuar una aplicación académica ya existente para el modelado de diagramas Entidad-Relación. En lugar de comenzar desde cero, el proyecto permite trabajar sobre una base previa, analizar sus limitaciones y evolucionarla poco a poco, manteniendo una herramienta accesible desde el navegador y orientada principalmente a un uso académico.

Además, el proyecto permite relacionar dos aspectos importantes del diseño de bases de datos: la representación conceptual mediante diagramas y su posterior transformación hacia un esquema relacional. Por ello, el trabajo no se centra únicamente en la edición gráfica, sino también en la coherencia entre el diagrama construido, las validaciones aplicadas y la salida SQL generada.

1.3. Punto de partida

El proyecto parte de una aplicación web previa orientada al modelado de diagramas Entidad-Relación, desarrollada con React y la librería mxGraph. Esta aplicación ya contaba con una base funcional para trabajar con entidades, atributos y relaciones, además de incluir mecanismos de validación y generación de SQL.

A partir de ese punto, este Trabajo Fin de Grado se centra en analizar el funcionamiento interno del editor y continuar su evolución. Esto implica trabajar sobre código ya existente, comprender decisiones de diseño heredadas y ampliar el soporte del sistema sin perder la relación entre la representación visual del diagrama y el modelo interno que maneja la aplicación.

Por tanto, el trabajo realizado no consistió en construir el editor desde cero, sino en comprender su funcionamiento interno, detectar limitaciones y evolucionarlo de forma incremental.

1.4. Problema abordado

El principal problema abordado en este trabajo es la evolución de un editor de diagramas Entidad-Relación ya existente. En una herramienta de este tipo no basta con representar elementos en pantalla, ya que cada entidad, atributo o relación debe tener también un significado dentro del modelo interno de la aplicación.

Por este motivo, cualquier ampliación del editor debe tener en cuenta varios aspectos al mismo tiempo: el lienzo visual, los datos que se almacenan internamente, las reglas de validación, la transformación posterior al modelo relacional y la generación del script SQL. El trabajo se centra en avanzar en esa evolución de forma controlada, revisando el comportamiento del sistema y evitando separar la parte gráfica de la parte conceptual.

1.5. Alcance del trabajo

El alcance de este Trabajo Fin de Grado se centra en la evolución de un editor web de diagramas Entidad-Relación ya existente. El proyecto no parte de una aplicación desarrollada desde cero, sino de una herramienta heredada que ya ofrecía una base funcional para la creación de diagramas, su validación y la generación de SQL.

A partir de ese punto, el trabajo realizado incluye el análisis del sistema previo, la estabilización de partes relevantes del editor y la ampliación controlada de sus capacidades. El eje principal del desarrollo ha sido mantener la coherencia entre lo que el usuario diseña en el lienzo, la representación interna del diagrama, las validaciones aplicadas, la transformación al modelo relacional y la generación de SQL.

Dentro del alcance implementado se incluyen mejoras en la representación interna del diagrama, en las reglas de validación, en la transformación al modelo relacional y en la generación del script SQL, así como la incorporación de nuevos elementos del modelo Entidad-Relación soportados por la aplicación. También se han añadido mejoras de interacción y usabilidad orientadas a facilitar el trabajo con el editor, como la selección múltiple, la ayuda integrada, opciones de importación y exportación, y ajustes visuales de la interfaz.

No se pretende construir una herramienta profesional completa de diseño de bases de datos ni cubrir todas las variantes posibles del modelo Entidad-Relación. Quedan fuera del alcance implementado funcionalidades avanzadas como las agregaciones, la cobertura completa de jerarquías ISA, la herencia múltiple real, la gestión de usuarios, la persistencia remota o la configuración detallada de tipos de datos SQL. Estas limitaciones se consideran decisiones de alcance del proyecto y posibles líneas de trabajo futuro.

1.6. Estructura de la memoria

La memoria se organiza en varios capítulos que presentan de forma progresiva el contexto del proyecto, los objetivos perseguidos, la base teórica necesaria y el trabajo realizado durante la evolución del editor. La estructura seguida es la siguiente:

- **Introducción:** presenta el contexto general del trabajo, la motivación, el punto de partida, el problema abordado y el alcance del proyecto.
- **Objetivos del proyecto:** recoge el objetivo general del TFG y los objetivos específicos que han guiado el desarrollo.
- **Conceptos teóricos:** introduce los conceptos del modelo Entidad-Relación y del modelo relacional necesarios para comprender el resto del documento.
- **Técnicas y herramientas:** describe las principales tecnologías y herramientas utilizadas durante el desarrollo del proyecto.

- **Aspectos relevantes del desarrollo:** explica las decisiones, cambios y problemas más importantes abordados durante la evolución del editor.
- **Trabajos relacionados:** presenta otras herramientas o enfoques relacionados con la creación de diagramas y el modelado de bases de datos.
- **Conclusiones y líneas de trabajo futuras:** resume los resultados obtenidos y plantea posibles mejoras o ampliaciones posteriores.

1.7. Documentos adjuntos

Junto con esta memoria se entregan varios documentos y recursos asociados al Trabajo Fin de Grado. Estos materiales complementan la explicación del proyecto y permiten revisar tanto el resultado obtenido como los elementos necesarios para su ejecución y evaluación.

- **Memoria del proyecto:** documento principal del TFG, en el que se presenta el contexto, los objetivos, la base teórica, el desarrollo realizado y las conclusiones obtenidas.
- **Anexos:** documento complementario que recoge información más detallada sobre la planificación, los requisitos, el diseño, la documentación de usuario y otros aspectos asociados al proyecto.
- **Aplicación web desplegada:** versión funcional de UBU E-R App publicada en Vercel, accesible desde el navegador para facilitar su revisión sin necesidad de instalar el proyecto en local. Acceso: [aplicación desplegada en Vercel](#).
- **Repositorio Git del proyecto:** repositorio que contiene el código fuente de la aplicación, la evolución del desarrollo y los recursos necesarios para su instalación y ejecución. Acceso: [repositorio del proyecto](#).
- **Instrucciones de instalación y ejecución:** indicaciones incluidas en el repositorio para preparar el entorno local, instalar dependencias, ejecutar la aplicación y lanzar las comprobaciones disponibles.
- **Ayuda integrada en la aplicación:** información accesible desde la propia interfaz del editor para orientar al usuario en el uso básico de la herramienta y en algunas operaciones relevantes.

2. Objetivos del proyecto

2.1. Objetivos generales

El objetivo principal de este Trabajo Fin de Grado es analizar y continuar el desarrollo de una aplicación web ya existente para el diseño de diagramas Entidad-Relación. El trabajo no parte de una herramienta creada desde cero, sino de un editor heredado que ya contaba con una base funcional y que se toma como punto de partida para introducir mejoras, ampliar sus capacidades y reforzar su coherencia interna.

Esta evolución se plantea manteniendo la relación entre las distintas partes de la aplicación. Por un lado, el usuario trabaja con una representación visual del diagrama en el lienzo. Por otro, la aplicación mantiene un modelo interno que debe reflejar correctamente lo que se ha diseñado. A partir de ese modelo se aplican validaciones y, cuando corresponde, se realizan transformaciones hacia el modelo relacional y la generación de SQL.

Por tanto, el objetivo general no se limita a añadir nuevas opciones al editor, sino a hacer que estas encajen dentro del funcionamiento global de la herramienta. Para ello resulta necesario comprender el sistema heredado, revisar sus decisiones de diseño y continuar su desarrollo de forma progresiva y controlada.

2.2. Objetivos específicos

A partir del objetivo general anterior, se plantean los siguientes objetivos específicos:

- Analizar el funcionamiento del editor heredado, identificando su estructura principal, las funcionalidades existentes y las decisiones de diseño que condicionan su evolución.
- Revisar y estabilizar la relación entre los elementos visuales del lienzo y la representación interna del diagrama, evitando inconsistencias entre lo que el usuario ve y lo que la aplicación procesa.
- Ampliar la representación interna de los diagramas Entidad-Relación para soportar nuevos elementos y propiedades, manteniendo la compatibilidad con la persistencia, la importación y la exportación de diagramas.
- Incorporar soporte para elementos adicionales del modelo Entidad-Relación, como entidades débiles, dependencias por identificación, atributos compuestos, atributos multivaluados, relaciones ternarias con roles y un soporte inicial para jerarquías ISA.
- Mantener la coherencia entre la parte visual del editor, las reglas de validación, la transformación al modelo relacional y la generación de SQL, de forma que los elementos soportados puedan traducirse de manera razonable a tablas, columnas, claves primarias y claves foráneas.
- Mejorar la validación del diagrama y la comunicación de errores al usuario, haciendo que los mensajes sean más claros y, cuando sea posible, vinculados a los elementos concretos relacionados con cada problema.
- Incorporar mejoras de usabilidad y apoyo a la edición, incluyendo una organización más clara de la interfaz, acciones contextuales más comprensibles, selección múltiple, ayuda integrada, soporte básico de idioma y operaciones de importación, exportación y generación de estructuras de ejemplo.
- Comprobar el comportamiento de la aplicación mediante pruebas automatizadas cuando proceda y mediante revisión manual en aquellos casos que requieran validación funcional o visual.
- Documentar el trabajo realizado, tanto desde el punto de vista del desarrollo como desde el uso de la aplicación y la evolución seguida durante el proyecto.

3. Conceptos teóricos

3.1. Modelo Entidad-Relación

El modelo Entidad-Relación es un modelo conceptual utilizado para representar la información de un dominio antes de llevarla a una base de datos concreta. Fue propuesto por Chen como una forma de describir la información mediante entidades y relaciones, separando el diseño conceptual de los detalles propios de la implementación final [2].

Esta separación resulta útil porque permite razonar primero sobre el problema que se quiere representar, sin empezar directamente por tablas, columnas o sentencias SQL. De esta forma, el diseño puede centrarse en identificar qué elementos intervienen, qué información se almacena sobre ellos y cómo se relacionan entre sí.

En el contexto de este trabajo, el modelo Entidad-Relación sirve como base teórica para el editor desarrollado. La aplicación permite representar visualmente diagramas E/R y, a partir de esa representación, aplicar validaciones y transformaciones posteriores hacia el modelo relacional. Por ello, los elementos gráficos del editor no deben entenderse solo como dibujos, sino como partes de un modelo con significado propio.

La Figura 3.1 muestra un ejemplo sencillo de diagrama Entidad-Relación, en el que se representan entidades, atributos, claves, una interrelación y sus cardinalidades.

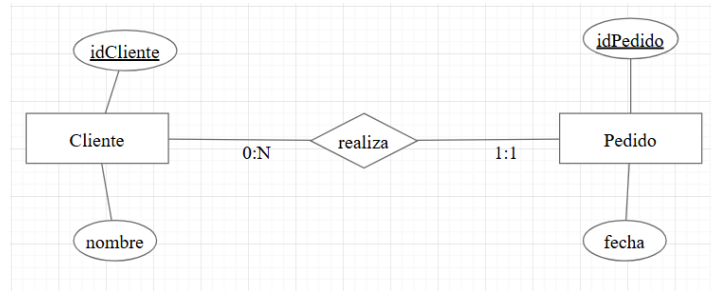


Figura 3.1: Ejemplo básico de diagrama Entidad-Relación con entidades, atributos, claves, una interrelación y cardinalidades.

3.2. Elementos básicos del modelo

Un diagrama Entidad-Relación se construye a partir de varios elementos que permiten describir la información de un dominio de forma visual. Aunque existen distintas notaciones y variantes, en este trabajo se consideran especialmente los conceptos que son necesarios para entender el funcionamiento del editor y las transformaciones que se realizan posteriormente [11].

Entidades y atributos

Las entidades representan objetos o conceptos del dominio sobre los que se quiere almacenar información. Por ejemplo, en un sistema académico podrían existir entidades como alumno, asignatura o profesor. Cada entidad agrupa un conjunto de elementos del mismo tipo y se describe mediante atributos.

Los atributos representan propiedades de una entidad o de una interrelación. Algunos atributos pueden actuar como identificadores, mientras que otros simplemente aportan información adicional. En el modelo relacional, los atributos identificadores tienen una relación directa con las claves primarias, ya que permiten distinguir de forma única las ocurrencias de una entidad.

Además de los atributos simples, el modelo Entidad-Relación puede incluir atributos con una estructura más compleja, como ocurre con los atributos compuestos o multivaluados. Estos casos se tratan con más detalle en el apartado de elementos avanzados considerados.

Interrelaciones, cardinalidades y roles

Las interrelaciones representan asociaciones entre entidades. Permiten expresar que dos o más entidades participan en un mismo hecho del dominio. Por ejemplo, una entidad alumno puede relacionarse con una entidad asignatura mediante una interrelación de matrícula.

Las cardinalidades indican la forma en la que las entidades participan en una interrelación. Esta información es importante porque condiciona la interpretación del diagrama y también su transformación posterior al modelo relacional. No se representa igual una relación uno a muchos que una relación muchos a muchos, ya que la estructura relacional resultante puede ser distinta.

En algunos casos, una misma entidad puede participar más de una vez en una interrelación. Para evitar ambigüedades, se utilizan roles, que indican el papel que desempeña cada participación. Esto es especialmente útil en interrelaciones reflexivas o en relaciones de mayor grado donde un mismo conjunto de entidades aparece repetido.

3.3. Elementos avanzados considerados

Además de los elementos básicos, el modelo Entidad-Relación permite representar situaciones más complejas. En este trabajo no se pretende cubrir todas las variantes posibles del modelo, pero sí se han tenido en cuenta algunos elementos avanzados que resultan importantes para el editor desarrollado.

Entidades débiles

Una entidad débil es una entidad que no puede identificarse completamente por sí misma y depende de otra entidad para formar su identificación. Esta dependencia suele representarse mediante una relación identificadora entre la entidad débil y la entidad propietaria.

En estos casos, la clave de la entidad débil se construye normalmente a partir de la clave de la entidad fuerte y de un atributo propio que actúa como discriminante. Por este motivo, las entidades débiles tienen un tratamiento especial tanto en la validación del diagrama como en la transformación al modelo relacional [11].

La Figura 3.2 resume este patrón: la entidad débil se identifica mediante la clave de la entidad propietaria y un discriminante propio.

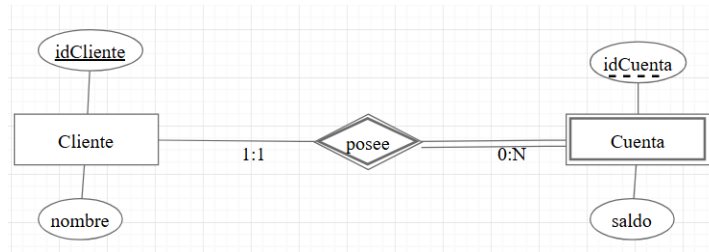


Figura 3.2: Ejemplo de entidad débil asociada a una entidad propietaria mediante una relación identificadora.

Atributos compuestos y multivaluados

Los atributos compuestos permiten representar información que puede dividirse en partes. Por ejemplo, una dirección podría descomponerse en calle, número, ciudad y código postal. Esta descomposición ayuda a expresar mejor la estructura de la información en el modelo conceptual.

Los atributos multivaluados representan propiedades que pueden tener varios valores para una misma entidad. Por ejemplo, una persona podría tener varios teléfonos. En el paso al modelo relacional, este tipo de atributo no suele representarse como una única columna de la tabla principal, sino mediante una estructura separada que permita almacenar varios valores.

Interrelaciones ternarias

Las interrelaciones ternarias son relaciones en las que participan tres entidades. No siempre pueden sustituirse por varias relaciones binarias sin perder significado, ya que en algunos casos el hecho que se quiere representar depende de la combinación de los tres participantes [11].

Este tipo de relación requiere prestar atención a las cardinalidades y a los posibles roles de los participantes. En el editor desarrollado, las relaciones ternarias son relevantes porque obligan a conservar más información en el modelo interno y a tratarlas de forma específica durante la transformación relacional.

La Figura 3.3 muestra un caso en el que la interrelación depende de tres participantes y no de asociaciones binarias independientes.

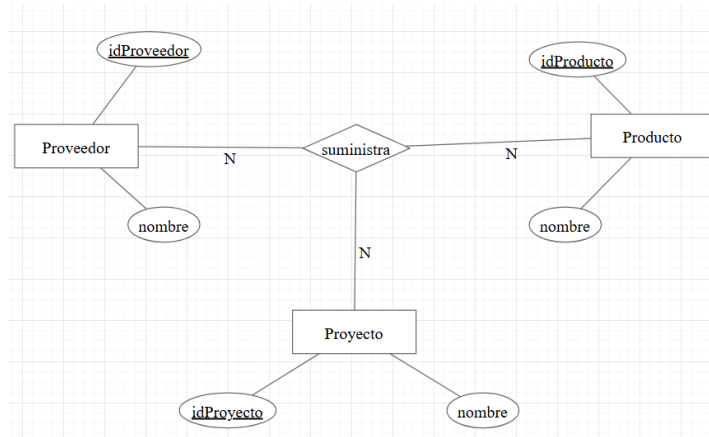


Figura 3.3: Ejemplo de interrelación ternaria con tres entidades participantes y cardinalidades.

Jerarquías ISA

Las jerarquías ISA representan relaciones de generalización y especialización entre entidades. Una entidad general recoge características comunes, mientras que las especializaciones representan casos más concretos. Por ejemplo, una entidad persona podría especializarse en alumno y profesor.

En este tipo de jerarquías, las especializaciones heredan la identificación de la entidad general. Además, pueden existir variantes avanzadas, como jerarquías totales o parciales, solapadas o exclusivas, o con discriminantes [11]. Sin embargo, no siempre es necesario implementar todas estas posibilidades para que el modelo sea útil.

En este trabajo se considera un soporte inicial de ISA. La idea principal es permitir representar una generalización con una o varias especializaciones y trasladar esa estructura al modelo relacional de forma controlada.

Como se aprecia en la Figura 3.4, una jerarquía ISA separa la entidad general de sus especializaciones, que heredan su identificación.

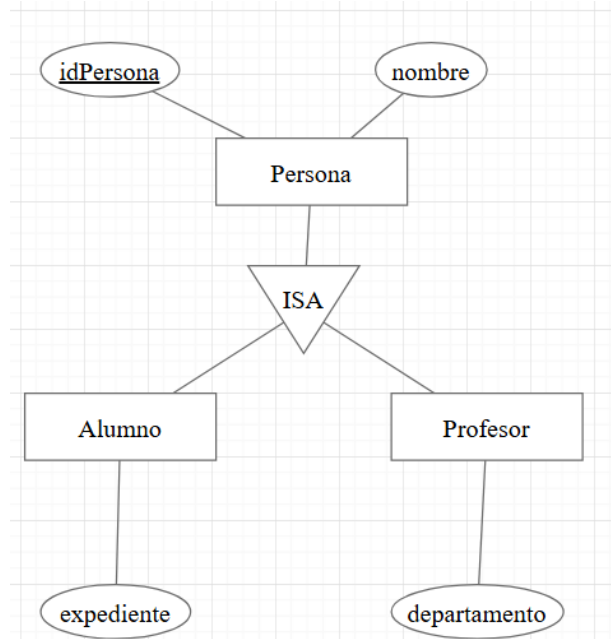


Figura 3.4: Ejemplo de jerarquía ISA con una entidad general y dos especializaciones.

Agregaciones

Las agregaciones permiten tratar una interrelación como un elemento de nivel superior dentro del modelo Entidad-Relación, de forma que pueda participar en otra relación con nuevos elementos del dominio [11]. Su incorporación a un editor visual requiere decisiones específicas sobre representación gráfica, validación, persistencia y transformación al modelo relacional.

En este trabajo las agregaciones se consideran únicamente como parte del contexto teórico del modelo, pero no forman parte del alcance implementado. Su soporte se deja como una posible línea de trabajo futura.

3.4. Transformación al modelo relacional

El modelo Entidad-Relación permite describir un dominio desde un punto de vista conceptual, pero para poder implementarlo en una base de datos relacional es necesario transformarlo a una estructura formada por tablas, columnas y relaciones entre ellas. Esta transformación sirve como paso intermedio entre el diseño conceptual y la generación final de sentencias SQL.

De forma general, las entidades del diagrama suelen convertirse en tablas, sus atributos pasan a ser columnas y los atributos identificadores se utilizan para definir claves primarias. Las interrelaciones entre entidades condicionan la forma en la que se crean claves foráneas o tablas intermedias, dependiendo de la cardinalidad y de la participación de cada entidad en la relación.

Las relaciones uno a muchos suelen representarse incorporando una clave foránea en la tabla situada en el lado muchos de la relación. En las relaciones muchos a muchos, normalmente se crea una tabla intermedia que contiene las claves de las entidades participantes y, si existen, los atributos propios de la relación.

Algunos elementos requieren reglas específicas. Los atributos multivaluados pueden necesitar tablas propias; las entidades débiles incorporan la clave de su entidad propietaria como parte de su identificación; y las relaciones ternarias suelen transformarse mediante una tabla que referencia a las entidades participantes.

En el caso de las jerarquías ISA, existen varias estrategias posibles de transformación. En este trabajo, para el soporte inicial implementado, se adopta una estrategia sencilla: la entidad general se transforma como una tabla ordinaria y cada especialización se transforma en una tabla propia que incorpora la clave de la generalización como clave primaria y clave foránea [11]. De esta forma, la especialización queda vinculada con la entidad general mediante una relación uno a uno.

La Figura 3.5 recoge la estrategia aplicada en este trabajo para transformar una jerarquía ISA en tablas: una tabla para la generalización y una tabla por cada especialización.

```
PERSONA
-----
idPersona  PK
nombre

ALUMNO
-----
idPersona  PK, FK → PERSONA(idPersona)
expediente

PROFESOR
-----
idPersona  PK, FK → PERSONA(idPersona)
departamento
```

Figura 3.5: Transformación relacional de una jerarquía ISA mediante tabla de generalización y tabla por especialización.

Esta fase es especialmente importante en el contexto del proyecto porque actúa como puente entre el diagrama visual diseñado por el usuario y la salida SQL generada por la aplicación.

3.5. Generación de SQL

SQL es el lenguaje utilizado habitualmente para definir y manipular bases de datos relacionales. Dentro del contexto de este trabajo, interesa especialmente la parte relacionada con la definición de estructuras, como la creación de tablas, claves primarias y claves foráneas.

Una vez que un diagrama Entidad-Relación se ha transformado en un modelo relacional, es posible generar sentencias SQL que representen esa estructura. En este proceso, las tablas resultantes se convierten en sentencias `CREATE TABLE`, los atributos se traducen a columnas y las relaciones entre tablas se expresan mediante claves foráneas.

En el editor desarrollado, la generación de SQL se entiende como una fase posterior al diseño y a la validación del diagrama. Por tanto, el resultado generado depende de que el modelo interno conserve correctamente la información necesaria sobre entidades, atributos, claves, interrelaciones y jerarquías ISA. De esta forma, el SQL no se genera únicamente a partir de

la parte visual del diagrama, sino a partir de la interpretación interna que la aplicación mantiene de él.

En la herramienta, esta generación se realiza intentando mantener un resultado legible y coherente con el modelo relacional obtenido. Por ejemplo, cuando una clave foránea referencia la clave primaria completa de otra tabla, la salida puede expresarse de forma compacta sin repetir información que ya queda definida en la tabla referenciada.

El SQL generado debe entenderse como una traducción razonable del modelo diseñado, adecuada para los objetivos del proyecto. No pretende sustituir el trabajo completo de diseño físico de una base de datos real, donde podrían decidirse tipos de datos concretos, índices, restricciones adicionales u optimizaciones específicas.

3.6. Alcance teórico aplicado en el proyecto

El modelo Entidad-Relación incluye más posibilidades de las que se han incorporado en la herramienta. Por ello, este capítulo se ha centrado en los conceptos que resultan necesarios para entender la evolución realizada: entidades, atributos, relaciones binarias y ternarias, entidades débiles, atributos compuestos y multivaluados, transformación al modelo relacional, generación de SQL y un soporte inicial para jerarquías ISA.

En el caso de las jerarquías ISA, el alcance aplicado se limita a una generalización con una o varias especializaciones, junto con la herencia de clave hacia las tablas especializadas. No se incluyen variantes más avanzadas como totalidad o parcialidad, especializaciones solapadas o exclusivas, discriminantes ni herencia múltiple real.

Las agregaciones se han tenido en cuenta como concepto teórico, pero no forman parte del alcance implementado. Esta delimitación permite ajustar el desarrollo al objetivo principal del trabajo: evolucionar el editor heredado manteniendo la coherencia entre el diagrama visual, el modelo interno, la validación, la transformación relacional y la generación de SQL.

4. Técnicas y herramientas

4.1. Entorno de desarrollo web

El proyecto se desarrolla como una aplicación web ejecutada en el navegador. Este enfoque permite que el editor pueda utilizarse sin instalar una aplicación de escritorio específica, manteniendo gran parte de la lógica dentro del propio cliente web.

La aplicación está desarrollada principalmente en JavaScript y se apoya en Node.js y npm para la gestión de dependencias y la ejecución de los scripts del proyecto [17, 16]. A través de estos scripts se realizan tareas habituales como arrancar la aplicación en modo desarrollo, generar la versión de producción o ejecutar las pruebas definidas.

Durante el desarrollo se utilizó Visual Studio Code como editor principal [14]. Esta herramienta permitió trabajar de forma cómoda con el código fuente, revisar la estructura del proyecto, utilizar integración con Git y modificar tanto los ficheros de implementación como los documentos asociados al trabajo.

Este planteamiento encaja con el tipo de herramienta desarrollada, ya que el usuario interactúa con una interfaz gráfica y un área de edición visual que deben mantenerse coordinadas con el modelo interno de la aplicación.

4.2. Librerías principales del editor

La interfaz del proyecto se desarrolla utilizando React, una librería de JavaScript orientada a la construcción de interfaces de usuario mediante componentes [12]. En este trabajo, React sirve como base para organizar la

aplicación, gestionar sus distintas partes visuales y coordinar la interacción del usuario con el editor.

Para la representación y manipulación de los diagramas se utiliza mx-Graph [8]. Esta librería proporciona la base gráfica necesaria para trabajar con vértices, aristas y eventos dentro del área de edición. En el contexto del proyecto, mxGraph no se utiliza únicamente para dibujar elementos, sino también como parte del flujo que relaciona la representación visual del diagrama con el modelo interno de la aplicación.

Además, el proyecto utiliza Material UI para algunos componentes de la interfaz [15]. Esta librería facilita la creación de elementos visuales comunes, como botones, diálogos o controles de interacción, manteniendo una apariencia más uniforme dentro de la aplicación.

También se emplean dependencias de apoyo para aspectos concretos de la interfaz, como la visualización de notificaciones al usuario tras determinadas acciones. Estas dependencias no constituyen el núcleo del editor, pero contribuyen a mejorar la respuesta visual de la aplicación durante su uso.

En conjunto, estas librerías permiten construir un editor web en el que la interfaz, el área gráfica y las acciones del usuario se integran para representar diagramas Entidad-Relación de forma coherente.

4.3. Herramientas de validación y pruebas

El proyecto utiliza distintas herramientas para revisar el comportamiento de la aplicación y mantener una calidad mínima en el código. Estas herramientas no sustituyen a la revisión manual del editor, especialmente en los aspectos visuales o de interacción, pero ayudan a detectar errores y regresiones durante el desarrollo.

Para las pruebas unitarias se utiliza Vitest [20]. Esta herramienta permite comprobar partes concretas de la lógica de la aplicación, como funciones de validación, transformación o generación de resultados, sin necesidad de ejecutar manualmente todos los flujos desde la interfaz.

Además, el proyecto cuenta con pruebas end-to-end mediante Playwright [13]. Estas pruebas permiten ejecutar escenarios completos sobre la aplicación en el navegador, simulando acciones del usuario y comprobando que determinados flujos siguen funcionando correctamente.

Junto a las pruebas, se utiliza Biome como herramienta de formateo y análisis estático del código [1]. Su uso ayuda a mantener un estilo más

uniforme y a detectar ciertos problemas antes de integrar cambios en el proyecto. También se emplea Lefthook para ejecutar comprobaciones asociadas al flujo de trabajo con Git [10].

4.4. Control de versiones, integración y despliegue

Para el control de versiones se utiliza Git, lo que permite registrar los cambios realizados en el proyecto y trabajar de forma más ordenada durante el desarrollo [5]. El uso de commits y ramas facilita separar las distintas tareas, revisar la evolución del código y mantener un historial del trabajo realizado.

El proyecto se gestiona mediante GitHub, que se utiliza como repositorio remoto y como apoyo para la organización del desarrollo [6]. A través de sus herramientas se documentan tareas, se agrupa el trabajo en issues y milestones, y se mantiene una trazabilidad básica de la evolución del editor.

También se utilizaron mecanismos de comprobación asociados al repositorio mediante GitHub Actions. En concreto, estos flujos permiten ejecutar pruebas y revisiones de calidad del código, ayudando a detectar problemas antes de integrar cambios y contribuyendo a mantener la estabilidad durante la evolución incremental de la aplicación.

Además del entorno local, se utilizó Vercel para disponer de versiones desplegadas de la aplicación durante el desarrollo [19]. Estos despliegues permitieron validar prototipos funcionales desde el navegador y facilitar la revisión de cambios sin depender siempre de una instalación local del proyecto.

4.5. Documentación y redacción de la memoria

Además de las herramientas relacionadas con el código, durante el proyecto se ha trabajado con documentación en LaTeX. La memoria y los anexos siguen la estructura de la plantilla del Trabajo Fin de Grado.

Para la redacción se utiliza Overleaf como entorno de trabajo del proyecto LaTeX [18]. Esta herramienta permite editar la documentación de forma progresiva y comprobar que la memoria y los anexos compilan correctamente mientras se van completando los distintos capítulos.

Esta parte documental permite justificar el proceso seguido, las decisiones tomadas, el alcance real de la implementación y las posibles líneas de continuación.

La Tabla 4.1 resume el papel principal de las herramientas utilizadas durante el desarrollo y la documentación del proyecto.

Tabla 4.1: Resumen de herramientas utilizadas en el proyecto.

Herramienta	Uso en el proyecto
Node.js y npm	Gestión de dependencias y ejecución de scripts del proyecto.
Visual Studio Code	Editor principal utilizado durante el desarrollo.
React	Construcción de la interfaz web mediante componentes.
mxGraph	Representación y manipulación gráfica de los diagramas.
Material UI	Componentes visuales de apoyo para la interfaz.
Vitest	Pruebas unitarias de partes concretas de la lógica.
Playwright	Pruebas end-to-end sobre la aplicación en navegador.
Biome	Formateo y análisis estático del código.
Lefthook	Ejecución de comprobaciones asociadas al flujo de trabajo con Git.
Git y GitHub	Control de versiones, organización de tareas y trazabilidad.
GitHub Actions	Ejecución automática de comprobaciones, pruebas y revisiones de calidad del código.
Vercel	Despliegue de versiones funcionales de la aplicación para validación y revisión desde el navegador.
Overleaf y LaTeX	Redacción y compilación de la memoria y los anexos.

5. Aspectos relevantes del desarrollo del proyecto

5.1. Planteamiento y organización del desarrollo

El desarrollo de este trabajo no comenzó desde una aplicación vacía, sino a partir de un editor de diagramas Entidad-Relación que ya existía y que tenía una parte funcional implementada. Antes de añadir nuevas funcionalidades, fue necesario entender cómo estaba construido el sistema, qué partes podían aprovecharse y qué aspectos convenía revisar para poder seguir ampliándolo con mayor seguridad.

Una de las ideas principales del proyecto ha sido mantener la coherencia entre las distintas partes del editor. No basta con que un elemento se dibuje correctamente en el lienzo, sino que también debe quedar representado de forma adecuada en el modelo interno de la aplicación. Ese mismo modelo es el que después se utiliza para validar el diagrama, transformarlo al modelo relacional y generar el script SQL correspondiente.

Por esta razón, el desarrollo se planteó de forma incremental. Primero se analizó el funcionamiento del proyecto heredado y se revisaron algunos puntos que podían dificultar futuras ampliaciones. Después se realizaron tareas de estabilización y reorganización del núcleo del editor. A partir de esa base se fueron incorporando nuevos elementos del modelo Entidad-Relación, intentando que cada ampliación quedara integrada tanto en la interfaz como en la validación, la transformación relacional y la generación de SQL.

En este capítulo se recogen los aspectos más relevantes de ese proceso. No se pretende hacer una descripción cronológica de todos los cambios realizados, sino explicar las decisiones técnicas más importantes y cómo estas permitieron evolucionar el editor desde su estado inicial hasta una versión más completa y ajustada a los objetivos del trabajo.

Organización del trabajo

El desarrollo se organizó de forma incremental, utilizando ramas, commits, issues y milestones de GitHub para separar las tareas y mantener trazabilidad entre los problemas detectados, las soluciones aplicadas y las comprobaciones posteriores.

Esta organización resultó especialmente útil al trabajar sobre una aplicación heredada, donde un cambio podía afectar simultáneamente a la interfaz, el modelo interno, la persistencia, la validación o la generación de SQL. Por ello, se intentó que cada bloque de trabajo pudiera revisarse antes de continuar con nuevas ampliaciones.

Además, algunas decisiones de alcance se ajustaron durante el desarrollo a partir de la revisión del tutor. Esto fue especialmente relevante en las fases finales, donde se decidió mantener un soporte inicial y controlado para las jerarquías ISA y aplazar funcionalidades más complejas, como las agregaciones o la herencia múltiple real.

5.2. Análisis y estabilización del sistema heredado

Antes de empezar con las ampliaciones del editor, se realizó una revisión del sistema heredado. La aplicación ya permitía trabajar con diagramas Entidad-Relación básicos y estaba desarrollada con React y mxGraph. Además, contaba con una estructura interna para guardar la información del diagrama, lo que resultaba fundamental para el resto de funcionalidades del proyecto.

Análisis del sistema heredado

Durante esta revisión se estudió la relación entre los elementos que aparecen en el lienzo y los datos que maneja internamente la aplicación. En particular, se analizó cómo se representaban las entidades, los atributos y las relaciones, cómo se actualizaba el modelo cuando el usuario modificaba

el diagrama y cómo se reconstruía el contenido al cargar un diagrama previamente guardado.

Con este análisis se vio que el editor funcionaba realmente en dos niveles. Por un lado estaba la parte visual, gestionada mediante `mxGraph`, que es la que permite mostrar y manipular los elementos en pantalla. Por otro lado estaba el modelo interno, que es el que utiliza la lógica de la aplicación para guardar el estado del diagrama, validarlo y transformarlo posteriormente a SQL.

Esta sincronización entre la parte visual y el modelo interno era uno de los puntos más importantes del proyecto. Si ambos niveles no se actualizan de forma coherente, puede ocurrir que el usuario vea un diagrama en pantalla que no coincida exactamente con la información que después se valida o se utiliza para generar SQL.

La revisión del sistema heredado también permitió detectar algunos puntos delicados de la implementación inicial. Algunas partes dependían de convenciones internas o de relaciones implícitas entre los elementos gráficos y los datos del modelo. Aunque estas soluciones podían funcionar en casos sencillos, podían complicar la incorporación de nuevos elementos del modelo Entidad-Relación. Por ello, antes de seguir ampliando la herramienta, fue necesario dedicar parte del trabajo a estabilizar el núcleo del editor.

Estabilización del núcleo del editor

Una vez comprendido el funcionamiento general del sistema heredado, el siguiente paso fue estabilizar algunas partes del núcleo del editor antes de incorporar nuevas funcionalidades. Esta fase fue importante porque el editor no solo debía permitir dibujar elementos en el lienzo, sino mantener una relación coherente entre esos elementos gráficos y la información almacenada internamente.

Por ello, una parte del trabajo se centró en reducir esas dependencias implícitas y hacer más explícitas algunas asociaciones internas. En particular, se revisó la forma en la que los elementos del modelo se vinculaban con las celdas de `mxGraph`, así como el modo en que se mantenían las referencias necesarias para reconstruir el diagrama tras una recarga o una importación.

También se revisó la persistencia de la información del diagrama. En una herramienta de este tipo, guardar y recuperar correctamente el estado del editor es fundamental, ya que el usuario puede crear un modelo, modificarlo, exportarlo o volver a cargarlo más adelante. Si la información persistida

no coincide con el estado real del lienzo, pueden aparecer errores en fases posteriores, especialmente en la validación o en la generación de SQL.

Otro aspecto relevante fue la estabilización de la configuración de relaciones. Las relaciones son uno de los elementos más delicados del editor, porque conectan varios elementos y además contienen información adicional, como los extremos participantes, los roles o las cardinalidades. A partir de esta idea, se revisaron los casos en los que una relación podía quedar incompleta o mal configurada, intentando evitar que el usuario pudiera confirmar estados que no fueran válidos para el modelo interno.

Esta fase no consistió en rehacer por completo la aplicación, sino en preparar una base más segura sobre la que seguir trabajando. El objetivo era que las ampliaciones posteriores no se apoyaran en comportamientos frágiles del sistema, sino en una estructura más clara y fácil de mantener. Esto permitió continuar con la incorporación de nuevos elementos del modelo Entidad-Relación con menor riesgo de introducir inconsistencias entre la interfaz, el modelo interno y las operaciones de validación o transformación.

5.3. Evolución del modelo interno y persistencia

Una parte importante del desarrollo fue la evolución del modelo interno utilizado por la aplicación para representar el diagrama. En el editor, los elementos no existen únicamente como figuras dibujadas en el lienzo, sino que también deben quedar guardados en una estructura de datos que permita trabajar con ellos desde la lógica de la aplicación.

Este modelo interno actúa como una representación del diagrama Entidad-Relación diseñado por el usuario. En él se almacenan las entidades, las relaciones, los atributos y el resto de elementos necesarios para reconstruir el diagrama, validarlo y transformarlo posteriormente al modelo relacional.

La Figura 5.1 resume la relación entre el lienzo visual del editor, el modelo interno y las operaciones que dependen de esa representación.

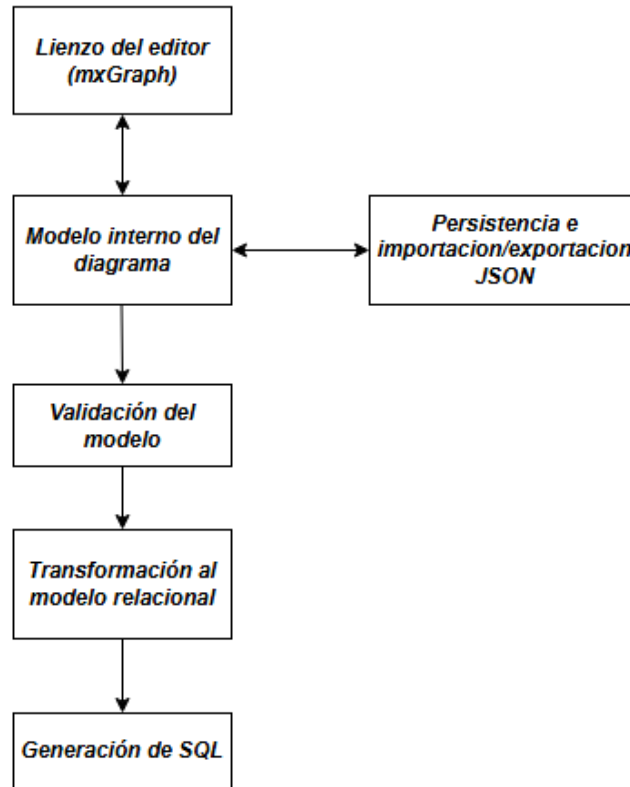


Figura 5.1: Relación entre el lienzo del editor, el modelo interno y las operaciones principales de la aplicación.

Por tanto, las ampliaciones del editor debían reflejarse también en esta estructura interna.

Evolución de la estructura interna

A medida que se añadieron nuevos elementos al editor, el modelo tuvo que incorporar más información. Por ejemplo, las entidades pasaron a poder almacenar si eran entidades débiles y, en ese caso, qué relación identificadora y qué entidad propietaria estaban asociadas a ellas. Del mismo modo, los atributos incorporaron información adicional para distinguir claves primarias, discriminantes, atributos multivaluados o atributos compuestos.

En el caso de las relaciones, también fue necesario ampliar la información almacenada. Además del nombre y de los extremos participantes, el modelo

debía conservar las cardinalidades, los roles cuando fueran necesarios y la posibilidad de que ciertas relaciones tuvieran atributos propios. Esta información era especialmente importante en relaciones más complejas, donde no basta con saber qué entidades participan, sino también cómo participa cada una de ellas.

Durante la fase final del desarrollo también se incorporó información específica para representar jerarquías ISA. Para ello, el modelo interno pasó a almacenar una colección de elementos ISA, cada uno de ellos formado por una entidad general y una o varias entidades especializadas. Esta información debía mantenerse separada de las entidades y relaciones ordinarias, pero al mismo tiempo debía estar conectada con ellas para poder validar el diagrama y generar posteriormente el esquema relacional.

Otro aspecto relevante fue la normalización de los datos del diagrama. Como el proyecto parte de una aplicación ya existente, era posible que algunos diagramas guardados previamente no tuvieran todos los campos que se fueron incorporando después. Para evitar errores al cargar estos datos, se añadió una fase de normalización que completa valores por defecto y adapta la información al formato esperado por la versión actual del editor.

Persistencia, importación y exportación

Otro aspecto importante del editor es la posibilidad de conservar el trabajo realizado por el usuario. Para ello, la aplicación utiliza una representación en formato JSON del modelo interno del diagrama. Esta representación permite guardar el estado del diagrama, exportarlo a un archivo e importarlo posteriormente.

La persistencia local permite que el usuario no pierda el contenido del editor al recargar la aplicación. Esta funcionalidad es especialmente útil durante el proceso de edición, ya que el diseño de un diagrama puede requerir varias iteraciones antes de considerarse terminado.

La exportación en JSON permite guardar el diagrama fuera del navegador y recuperarlo más adelante. A su vez, la importación permite cargar un diagrama previamente exportado. Antes de aceptar un archivo importado, la aplicación comprueba que su contenido sea válido y que pueda adaptarse al modelo interno esperado.

En las fases finales se amplió además el flujo de importación para permitir dos modos de trabajo. El usuario puede reemplazar el diagrama actual por el contenido del archivo importado, manteniendo el comportamiento

más simple y seguro, o combinar el diagrama importado con el contenido existente. En este segundo caso no se realiza una fusión semántica de elementos equivalentes, sino una inserción controlada basada en remapeo de identificadores, actualización de referencias internas, renombrado de conflictos y desplazamiento visual del bloque importado.

Esta parte fue especialmente relevante porque cada ampliación del modelo debía ser compatible con la persistencia. Si se añade un nuevo elemento al editor, como una relación ternaria o una jerarquía ISA, no basta con mostrarlo correctamente en pantalla: también debe poder guardarse, exportarse, importarse y reconstruirse de forma coherente.

De esta manera, la persistencia actúa como un punto de unión entre varias partes del sistema. El modelo interno no solo sirve para validar y generar SQL, sino también para mantener la continuidad del trabajo del usuario y permitir que los diagramas puedan moverse entre sesiones o entre distintos entornos.

5.4. Ampliación de los elementos Entidad-Relación soportados

Una vez estabilizada la base del editor, el desarrollo se centró en ampliar los elementos del modelo Entidad-Relación que podían representarse en la aplicación. Los elementos incorporados ya se han presentado desde el punto de vista teórico, por lo que en esta sección se explica cómo se integraron en el editor, atendiendo especialmente a su representación interna, validación, persistencia y transformación relacional.

Esta ampliación no consistió únicamente en añadir nuevas figuras al lienzo, sino en adaptar también el modelo interno, las validaciones y la transformación posterior al modelo relacional.

El punto de partida del editor ya permitía trabajar con elementos básicos, como entidades, atributos y relaciones binarias. A partir de esa base se fueron incorporando elementos que permiten representar modelos más completos, manteniendo siempre la correspondencia entre la parte visual y la información almacenada internamente por la aplicación.

Entidades débiles y dependencias por identificación

Una de las primeras ampliaciones relevantes fue el soporte para entidades débiles. Para ello, las entidades pasaron a poder marcarse como débiles y a

asociarse con una relación identificadora y una entidad propietaria. También se incorporó el concepto de discriminante, representado mediante atributos parciales de la entidad débil.

Esta ampliación afectó a varias partes del sistema. En la interfaz fue necesario diferenciar visualmente las entidades débiles y las relaciones identificadoras. En el modelo interno se añadieron los datos necesarios para recordar la dependencia entre la entidad débil y su propietaria. Además, la validación y la transformación relacional tuvieron que contemplar que la clave de una entidad débil se construye a partir de su discriminante y de la clave heredada de la entidad fuerte de la que depende.

Atributos compuestos y multivaluados

También se amplió el tratamiento de los atributos. Además de los atributos simples y las claves primarias, el editor pasó a permitir atributos compuestos y atributos multivaluados.

En el caso de los atributos compuestos, fue necesario representar una estructura jerárquica de atributos dentro del modelo interno. Esto permite que un atributo pueda actuar como padre de otros atributos más simples. Esta representación también debía poder reconstruirse visualmente al cargar un diagrama guardado.

Los atributos multivaluados requirieron un tratamiento diferente, ya que no se pueden trasladar al modelo relacional como una simple columna dentro de la tabla de la entidad. Por ello, su incorporación tuvo impacto no solo en la representación visual, sino también en la transformación relacional y en la generación del SQL final.

Relaciones ternarias y roles

Otro bloque importante fue la evolución de las relaciones. El editor ya trabajaba con relaciones binarias, pero durante el desarrollo se revisó su comportamiento y se amplió el soporte para casos más complejos. Entre ellos se encuentran las relaciones reflexivas, las relaciones con atributos propios cuando el tipo de relación lo permite y las relaciones ternarias.

En las relaciones ternarias, el modelo interno debía conservar tres participantes, sus cardinalidades y, cuando fuera necesario, los roles asociados a cada participación. El soporte para roles fue especialmente útil en los casos en los que una misma entidad participa más de una vez en una relación. En

esos casos, no basta con saber qué entidad interviene, sino que es necesario distinguir el papel que desempeña en cada lado de la relación.

Esta información resulta importante tanto para que el diagrama sea comprensible como para evitar ambigüedades durante la transformación posterior. Por este motivo, los roles se integraron en el modelo interno y en la representación visual de las relaciones ternarias.

Soporte inicial de jerarquías ISA

Durante la fase final del desarrollo se incorporó también un soporte inicial para jerarquías ISA. Esta ampliación se planteó con un alcance controlado, ya que las jerarquías de especialización y generalización pueden tener muchas variantes y no todas ellas eran necesarias para los objetivos del proyecto.

El editor permite crear un elemento ISA y configurarlo indicando una entidad general y una o varias entidades especializadas. Esta información se guarda en el modelo interno, se reconstruye al cargar el diagrama y se tiene en cuenta en la validación y en la generación del esquema relacional.

Una decisión importante fue limitar la funcionalidad a jerarquías ISA sencillas. Cada ISA debe tener una única generalización y una o varias especializaciones. No se incorporaron restricciones de totalidad o parcialidad, ni especializaciones solapadas o exclusivas, ni discriminantes, ni herencia múltiple real. Estas variantes se dejaron fuera para mantener una implementación coherente con el alcance del trabajo.

También se decidió que las especializaciones no tengan clave primaria propia, ya que heredan la clave de la generalización. Sin embargo, sí se permite que una especialización no tenga atributos propios, porque puede seguir siendo útil dentro del modelo si participa en relaciones con otras entidades.

5.5. Validación, transformación relacional y generación de SQL

Una vez que el usuario construye un diagrama en el editor, la aplicación debe comprobar que ese modelo es coherente antes de generar cualquier salida. Por ello, el flujo principal no termina en la representación visual del diagrama, sino que continúa con tres pasos relacionados entre sí: la validación del modelo, su transformación a una representación relacional y la generación final del script SQL.

Validación del modelo

La validación del modelo fue una parte importante del desarrollo, porque el editor no debía limitarse a permitir dibujar diagramas, sino que también debía comprobar que esos diagramas tuvieran una estructura coherente antes de realizar operaciones como la transformación relacional o la generación de SQL.

En una herramienta de este tipo, el usuario puede construir el diagrama de forma progresiva y dejar temporalmente elementos incompletos. Esto es normal durante la edición, pero no todos esos estados son válidos para generar un esquema relacional. Por este motivo, la aplicación necesita diferenciar entre un diagrama en construcción y un diagrama suficientemente completo para poder procesarse.

La validación se apoya en el modelo interno del diagrama. En lugar de revisar únicamente la parte visual, se comprueba la información almacenada sobre entidades, atributos, relaciones y jerarquías ISA. De esta forma, las reglas de validación se aplican sobre la misma estructura que después se utiliza para la transformación relacional y la generación de SQL.

Entre las comprobaciones realizadas se encuentran reglas generales, como evitar elementos sin nombre o nombres repetidos cuando pueden producir ambigüedad, y reglas más específicas del modelo Entidad-Relación. Por ejemplo, se revisa que las entidades tengan una definición suficiente, que los atributos clave estén correctamente indicados y que las relaciones tengan sus participantes y cardinalidades configurados de forma válida.

A medida que se ampliaron los elementos soportados por el editor, también fue necesario ampliar las validaciones. Las entidades débiles, los discriminantes, los atributos compuestos o multivaluados y las relaciones más complejas introducen restricciones adicionales que no aparecían en el modelo inicial. Por ello, la validación fue evolucionando junto con el resto del sistema.

En el caso de las jerarquías ISA, la validación comprueba que cada jerarquía tenga una única generalización y al menos una especialización. También se controla que una entidad no actúe como especialización en más de una ISA, ya que ese caso supondría herencia múltiple y se decidió dejarlo fuera del alcance implementado. Además, se evita que las especializaciones tengan claves primarias propias, puesto que heredan la clave de la entidad general.

El objetivo de estas comprobaciones no es sustituir a un sistema gestor de bases de datos ni cubrir cualquier posible restricción semántica, sino evitar

que la aplicación genere una salida incoherente a partir de un diagrama mal definido. En este sentido, la validación actúa como una barrera previa antes de pasar a fases posteriores del flujo.

También se procuró que los errores detectados pudieran comunicarse al usuario de forma comprensible. Esto es importante porque, si un diagrama no puede transformarse o generar SQL, el usuario necesita saber qué parte debe revisar. Por tanto, la validación no solo tiene una función interna, sino que también ayuda a guiar el uso correcto de la herramienta.

La Figura 5.2 muestra un ejemplo de los errores detectados durante la validación, presentados de forma agrupada para facilitar su revisión por parte del usuario.

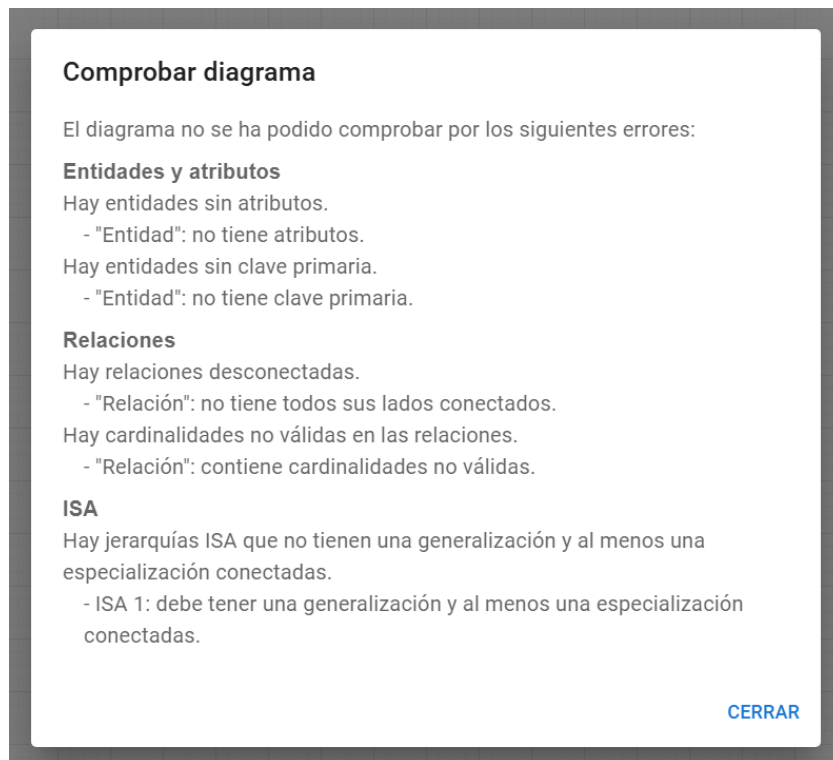


Figura 5.2: Diálogo de validación con errores agrupados del diagrama.

Transformación al modelo relacional

Después de validar el diagrama, el siguiente paso del flujo de la aplicación es transformarlo a una representación relacional. Esta fase es importante porque el modelo Entidad-Relación y el modelo relacional no representan la

información de la misma manera. Por ello, antes de generar el script SQL final, la aplicación necesita convertir las entidades, atributos y relaciones del diagrama en tablas, columnas, claves primarias y claves foráneas.

La transformación se realiza a partir del modelo interno del diagrama, no directamente desde los elementos gráficos del lienzo. Esto permite trabajar con una estructura más estable y centrada en la información del modelo, independientemente de cómo estén colocados visualmente los elementos en pantalla.

En primer lugar, las entidades del diagrama se convierten en tablas. Sus atributos simples se trasladan como columnas, y los atributos marcados como clave primaria pasan a formar parte de la clave principal de la tabla correspondiente. En el caso de atributos compuestos, se tiene en cuenta su estructura interna para obtener los atributos finales que deben aparecer en el esquema relacional.

Las relaciones se tratan en función de sus características y cardinalidades. En relaciones binarias, el proceso puede implicar la incorporación de claves foráneas en una de las tablas participantes o la creación de una tabla intermedia, dependiendo del tipo de relación. En las relaciones con atributos propios, esos atributos también deben conservarse durante la transformación para no perder información del modelo original.

Algunos elementos requieren un tratamiento específico. Por ejemplo, los atributos multivaluados no pueden representarse como una simple columna dentro de la tabla de la entidad, por lo que deben transformarse de manera separada. Del mismo modo, las entidades débiles necesitan conservar la dependencia respecto a su entidad propietaria y su discriminante, ya que esta información afecta a la construcción de su clave en el modelo relacional.

En el caso de relaciones más complejas, como las ternarias, la transformación debe conservar la información de los participantes y sus roles cuando son necesarios. Esto evita ambigüedades, especialmente cuando una misma entidad aparece más de una vez en una relación o cuando la relación necesita generar una estructura propia dentro del modelo relacional.

Para las jerarquías ISA se adoptó una estrategia sencilla y explícita. La entidad general se transforma como una tabla ordinaria. Cada especialización genera también su propia tabla, pero incorporando la clave de la generalización como clave primaria y, al mismo tiempo, como clave foránea hacia la tabla general. De esta forma, la relación entre la generalización y cada especialización queda representada en el modelo relacional mediante una relación uno a uno.

Generación de SQL

La generación de SQL es la última fase del flujo principal de la aplicación. Una vez que el diagrama ha sido validado y transformado a una representación relacional, el sistema utiliza esa información para construir un script SQL que pueda servir como punto de partida para crear las tablas correspondientes en una base de datos.

En el proyecto se decidió separar la transformación relacional de la generación del texto SQL. Esta separación permite que la aplicación primero trabaje con una estructura intermedia formada por tablas, columnas, claves primarias y claves foráneas, y que después otra parte del sistema se encargue de convertir esa estructura en sentencias SQL. De esta forma, el código queda más organizado y resulta más sencillo revisar si un error pertenece a la transformación del modelo o a la forma en la que se escribe el script final.

El script generado incluye las sentencias necesarias para crear las tablas obtenidas a partir del modelo relacional. En cada tabla se definen sus columnas, los tipos de datos utilizados, las claves primarias y las restricciones que correspondan. Además, cuando existen relaciones entre tablas, se generan también las claves foráneas necesarias para mantener la relación entre ellas.

Durante la revisión del generador se buscó producir un script más claro y legible. Para ello, las claves foráneas se incluyen dentro de las sentencias `CREATE TABLE` cuando las dependencias entre tablas pueden resolverse mediante el orden de creación. Las sentencias `ALTER TABLE` se mantienen como mecanismo de respaldo para los casos en los que aparecen dependencias cíclicas reales.

También se simplificó la representación de algunas claves foráneas cuando estas referencian la clave primaria completa de la tabla destino, evitando repetir información que ya queda definida en dicha tabla. Esta decisión no modifica la estructura relacional obtenida, sino la forma en la que se expresa el script final.

Asimismo, se realiza una normalización de los identificadores utilizados en el SQL. Esta normalización ayuda a evitar problemas con nombres que puedan contener espacios, acentos u otros caracteres poco adecuados para identificadores de base de datos. Además, cuando se detectan posibles repeticiones de nombres dentro de una misma tabla, se ajustan para evitar conflictos en el esquema resultante.

En el caso de las jerarquías ISA, el SQL generado refleja la estrategia relacional adoptada. La tabla correspondiente a la generalización se crea

como una tabla ordinaria. Las tablas de especialización incluyen la clave heredada de la generalización, que se declara como clave primaria y también como clave foránea hacia la tabla general. Con ello se mantiene la relación entre la entidad general y sus especializaciones dentro del esquema SQL.

El generador de SQL no pretende cubrir todos los detalles posibles de un sistema gestor de bases de datos, sino producir una salida coherente con el modelo relacional obtenido por la aplicación. Por ejemplo, los atributos se traducen con un tipo de dato general, suficiente para representar la estructura del esquema, pero sin entrar en una edición detallada de tipos específicos por atributo.

En conjunto, esta fase completa el recorrido desde el diagrama visual hasta una salida textual utilizable. El resultado no debe entenderse como un diseño de base de datos final para cualquier contexto real, sino como una traducción razonable del modelo creado en el editor, adecuada para el objetivo académico y práctico del proyecto.

La Figura 5.3 muestra un ejemplo de script SQL generado por la aplicación a partir de un diagrama previamente validado.

```
DROP TABLE IF EXISTS Entidad, Entidad_1, Relacion CASCADE;

CREATE TABLE Entidad (
  id VARCHAR(40) PRIMARY KEY,
  Atributo_1 VARCHAR(40)
);

CREATE TABLE Entidad_1 (
  id VARCHAR(40) PRIMARY KEY,
  Atributo_1 VARCHAR(40)
);

CREATE TABLE Relacion (
  id_Relacion_1 VARCHAR(40) REFERENCES Entidad,
  id_Relacion_2 VARCHAR(40) REFERENCES Entidad_1,
  Atributo VARCHAR(40),
  PRIMARY KEY (id_Relacion_1, id_Relacion_2)
);
```

Figura 5.3: Ejemplo de script SQL generado a partir de un diagrama validado.

5.6. Refinamientos de usabilidad del editor

Además de ampliar los elementos del modelo soportados por el editor, durante la fase final del trabajo se incorporaron varias mejoras orientadas a facilitar el uso de la herramienta. Estas mejoras no modifican la notación Entidad-Relación utilizada ni cambian la semántica del modelo, pero hacen que la interacción con el editor sea más clara, guiada y menos propensa a errores.

Organización de la interfaz y acciones contextuales

Una de las mejoras consistió en revisar la organización general de la interfaz. La barra lateral se reorganizó en secciones visuales para separar mejor las acciones de creación, las operaciones sobre el diagrama y las opciones de importación, exportación o generación. También se ajustó su presentación para mantener una anchura estable y evitar desplazamientos innecesarios de los controles.

Con esta reorganización se buscó que el usuario pudiera localizar las acciones principales de forma más rápida. Además, las acciones disponibles se presentan de forma más clara en función del elemento seleccionado. Por ejemplo, al seleccionar una entidad, una relación o un atributo, la interfaz muestra un encabezado contextual que ayuda a identificar sobre qué tipo de elemento se está trabajando.

La Figura 5.4 muestra un ejemplo de las acciones contextuales disponibles al seleccionar un elemento del diagrama, lo que permite concentrar las opciones relevantes según el tipo de elemento activo.

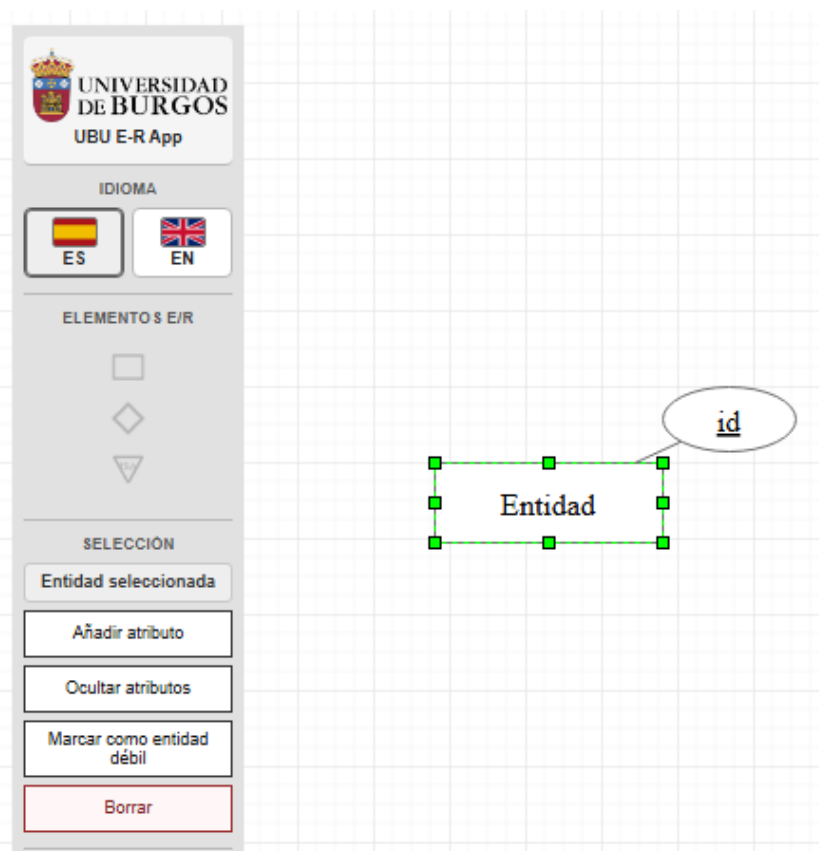


Figura 5.4: Acciones contextuales mostradas por el editor al seleccionar un elemento del diagrama.

También se añadieron pequeños textos de ayuda en zonas concretas del editor y se refinó el comportamiento de la selección múltiple. El rectángulo de selección se hizo visible y, cuando existen varios elementos seleccionados, la barra lateral evita mostrar acciones propias de un único elemento. En ese caso se mantienen únicamente acciones aplicables al conjunto seleccionado, como el desplazamiento en el lienzo o el borrado conjunto.

Mejora de validaciones, diálogos y mensajes de ayuda

Otro bloque de trabajo fue la revisión de los mensajes de validación y de los diálogos de confirmación. Antes de generar SQL y en determinadas operaciones del editor, la aplicación valida la estructura del modelo. En esta fase se mejoró la forma de presentar los errores detectados, agrupándolos

de manera más clara y mostrando, cuando es posible, el elemento concreto relacionado con el problema.

Esta mejora resulta útil porque el usuario no solo recibe una indicación genérica de que el diagrama no es válido, sino que puede identificar con mayor facilidad qué parte debe corregir. De esta forma, la validación funciona también como una ayuda durante el proceso de modelado.

Además, se revisaron los textos de confirmación de acciones sensibles, como generar SQL, importar un diagrama JSON o reiniciar el lienzo. También se añadieron textos breves de ayuda en los diálogos relacionados con jerarquías ISA y con roles en relaciones ternarias. Los diálogos más complejos pueden moverse por la pantalla, evitando que oculten de forma permanente la parte del diagrama que el usuario necesita consultar.

Operaciones globales sobre el diagrama

También se revisaron varias acciones que afectan al diagrama completo. Además de la importación y exportación en JSON, se añadió la posibilidad de exportar la representación visual del diagrama como imagen en formato PNG o SVG. Esta exportación se limita al aspecto visual del modelo y no modifica el JSON, la persistencia interna ni el SQL generado.

Se incorporó además una acción explícita para comprobar el diagrama sin generar SQL. De esta forma, el usuario puede revisar el estado del modelo directamente: si el diagrama es válido recibe una notificación de éxito, y si contiene errores se muestra el diálogo de validación agrupado.

Otra mejora fue la generación de estructuras básicas de ejemplo, como relaciones binarias, relaciones ternarias, entidades débiles o jerarquías ISA sencillas. Esta funcionalidad sirve como apoyo para probar la herramienta y crear modelos base, pero no pretende actuar como un sistema avanzado de plantillas ni realizar fusiones semánticas entre elementos equivalentes.

La Figura 5.5 muestra la generación de una estructura básica de ejemplo, utilizada como apoyo para crear rápidamente patrones habituales del modelo Entidad-Relación.

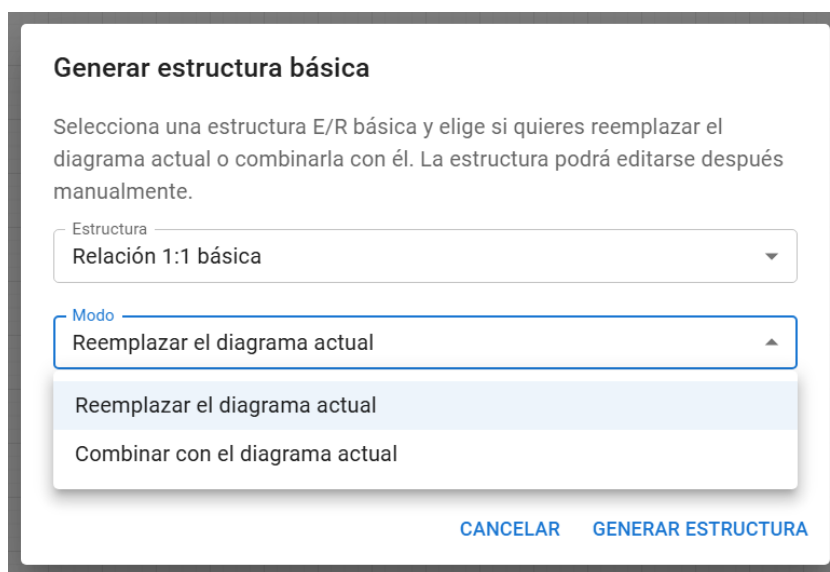


Figura 5.5: Generación de una estructura básica de ejemplo en el editor.

Por último, se añadieron mejoras de apoyo a la edición, como el ajuste de vista al contenido del diagrama, la ayuda integrada, la ventana de información del proyecto y un soporte básico de idioma español/inglés para los textos principales de la interfaz. Estas mejoras no cambian la notación académica del modelo Entidad-Relación, pero ayudan a que la herramienta resulte más clara durante la edición.

5.7. Pruebas y revisión manual

Además de las validaciones incorporadas en la propia aplicación, durante el desarrollo se utilizaron pruebas automáticas y revisión manual para comprobar el comportamiento del editor. Esta parte fue especialmente importante porque muchas funcionalidades no dependen solo de una función aislada, sino de la coordinación entre la interfaz, el modelo interno, la persistencia, la validación y la generación de resultados.

En el proyecto se trabajó con pruebas unitarias para comprobar partes concretas de la lógica de la aplicación. Este tipo de pruebas resulta útil para verificar funciones relacionadas con la validación del modelo, la transformación relacional o la generación de SQL, ya que permiten comprobar casos específicos sin necesidad de interactuar siempre con la interfaz gráfica.

También se utilizaron pruebas de extremo a extremo para revisar flujos completos de uso del editor. Estas pruebas permiten simular acciones del usuario sobre la aplicación, como crear elementos, modificar información, importar o exportar diagramas, generar SQL o comprobar que determinados comportamientos se mantienen después de interactuar con el lienzo. En un editor visual, este tipo de pruebas ayuda a detectar problemas que no siempre aparecen al probar únicamente funciones internas.

Aun así, no todo el comportamiento del editor puede comprobarse de forma sencilla mediante pruebas automáticas. Al tratarse de una aplicación visual, hay aspectos que requieren revisar manualmente el resultado en pantalla, como la colocación de elementos, la legibilidad de la representación gráfica, la claridad de los mensajes o la coherencia del diagrama después de varias operaciones seguidas.

Por eso, la revisión manual también tuvo un papel importante. Se comprobaron distintos casos de uso creando diagramas en el editor y revisando que los elementos se representaran correctamente, que pudieran guardarse y recuperarse, que las validaciones detectaran configuraciones incorrectas y que la salida relacional o SQL fuera coherente con el diagrama diseñado.

En las fases finales se prestó especial atención a los casos relacionados con las jerarquías ISA, las mejoras de usabilidad y la generación de SQL. En el caso de ISA, se revisó que las especializaciones heredasen correctamente la clave de la generalización, que se rechazasen configuraciones no soportadas y que el SQL generado mantuviera la relación entre las tablas de generalización y especialización. También se comprobaron mejoras de interfaz como los mensajes de validación agrupados, los diálogos de confirmación, el selector de idioma y la generación de estructuras básicas de ejemplo.

Esta combinación de pruebas automáticas y revisión manual permitió avanzar con más seguridad durante la ampliación del editor. Las pruebas ayudaron a controlar casos concretos y evitar regresiones, mientras que la revisión manual permitió comprobar el comportamiento global de la herramienta desde el punto de vista del usuario.

En cualquier caso, las pruebas no deben entenderse como una garantía absoluta de ausencia de errores. Su función dentro del proyecto fue apoyar el desarrollo y aportar confianza sobre los comportamientos principales, combinando comprobaciones automáticas con revisión manual en los escenarios más sensibles.

5.8. Dificultades y decisiones técnicas relevantes

Durante el desarrollo aparecieron varias dificultades que influyeron en la forma de ampliar el editor. Una de las más importantes fue mantener sincronizada la parte visual del diagrama con el modelo interno de la aplicación. `mxGraph` permite representar y mover elementos en el lienzo, pero la lógica del proyecto necesita una estructura propia para validar el diagrama, guardarlo, reconstruirlo y generar SQL.

Esta dificultad se hizo más visible al incorporar elementos más complejos que las entidades, atributos o relaciones binarias iniciales. Las entidades débiles, los atributos compuestos, las relaciones ternarias y las jerarquías ISA no solo requieren una representación gráfica específica, sino también información adicional en el modelo interno. Si esa información no se conserva correctamente, pueden aparecer incoherencias al guardar, importar, validar o transformar el diagrama.

También fue necesario tomar decisiones de alcance. El modelo Entidad-Relación permite representar casos avanzados, pero algunos de ellos podían aumentar demasiado la complejidad del proyecto. Esto ocurrió especialmente con las jerarquías ISA, donde se optó por implementar un soporte inicial y controlado. La herramienta permite una generalización y una o varias especializaciones, pero no incorpora totalidad o parcialidad, solapamiento o exclusividad, discriminantes ni herencia múltiple real. Estas variantes se dejaron fuera para evitar una implementación incompleta o difícil de justificar dentro del alcance del trabajo.

Una decisión similar se tomó con las agregaciones. Aunque forman parte del temario teórico del modelo Entidad-Relación, su incorporación requeriría nuevas decisiones sobre representación visual, modelo interno, validación, persistencia y transformación relacional. Por ello, se mantuvieron como una posible línea de trabajo futura y no como parte del alcance implementado.

Otra dificultad relevante apareció en la generación de SQL. En una primera aproximación, las claves foráneas podían generarse de forma separada mediante sentencias `ALTER TABLE`. Sin embargo, este enfoque no siempre produce el script más claro, ya que en muchos diagramas las dependencias entre tablas pueden resolverse simplemente creando las tablas en un orden adecuado. Por este motivo, se revisó el generador para incluir las claves foráneas dentro del `CREATE TABLE` cuando es posible, manteniendo `ALTER TABLE` únicamente como mecanismo de respaldo para ciclos reales de dependencias.

En las mejoras de usabilidad también fue necesario equilibrar utilidad y complejidad. Algunas mejoras, como reorganizar la barra lateral, añadir mensajes de ayuda o mejorar los diálogos de confirmación, tenían un riesgo bajo y aportaban claridad al editor. En cambio, otras posibilidades, como fusionar diagramas de forma semántica o copiar y pegar fragmentos completos del diagrama, podían introducir conflictos de identificadores, nombres repetidos o referencias internas rotas.

Por este motivo, la importación combinada de JSON y la inserción de estructuras de ejemplo se plantearon de forma controlada. El usuario puede elegir entre reemplazar el diagrama actual o insertar nuevo contenido en el diagrama existente. En el segundo caso, la aplicación remapea identificadores internos, actualiza referencias, resuelve conflictos de nombres mediante renombrado y desplaza visualmente el bloque insertado.

Esta solución no intenta realizar una fusión semántica avanzada entre diagramas, pero permite combinar contenido de forma práctica y coherente con el alcance del proyecto.

El soporte de idioma también se planteó con un alcance limitado. Se añadió traducción para los textos principales de la interfaz en español e inglés, pero no se intentó convertir la aplicación en un sistema de internacionalización completo. Esta decisión permitió mejorar la accesibilidad de la herramienta sin modificar la semántica del modelo ni alterar el contenido creado por el usuario.

En conjunto, las principales decisiones técnicas del proyecto estuvieron condicionadas por la necesidad de mantener la coherencia entre interfaz, modelo interno, validación, persistencia y generación SQL. En lugar de añadir funcionalidades de forma aislada, se priorizó que cada ampliación quedara integrada en el flujo completo de la aplicación.

6. Trabajos relacionados

En este capítulo se presentan algunas herramientas relacionadas con el modelado de diagramas y el diseño de bases de datos. La comparación no pretende valorar estas alternativas de forma absoluta, ya que muchas de ellas son más maduras y tienen objetivos más amplios que los de este Trabajo Fin de Grado. El objetivo es situar el proyecto desarrollado dentro de un conjunto de soluciones existentes y señalar sus diferencias principales.

Entre las herramientas consideradas se encuentran [diagrams.net](#), también conocida como [draw.io](#), [Excalidraw](#), [ERDPlus](#) y [dbdiagram.io](#) [9, 4, 3, 7]. Las dos primeras se orientan a la diagramación general, mientras que las dos últimas están más relacionadas con el modelado de bases de datos.

6.1. Herramientas generales de diagramación

[diagrams.net](#) / [draw.io](#)

[Draw.io](#), actualmente conocido también como [diagrams.net](#), es una aplicación web que permite crear diagramas de muchos tipos. Su principal ventaja es la flexibilidad, ya que permite dibujar estructuras muy variadas mediante formas, conectores y plantillas. Por ello, puede utilizarse para representar diagramas Entidad-Relación de forma visual. Sin embargo, su enfoque es generalista: los elementos dibujados no tienen necesariamente una semántica interna propia de un modelo E/R académico.

Esto marca una diferencia importante respecto al proyecto desarrollado. En el editor de este trabajo, los elementos del diagrama no se tratan solo como figuras, sino como entidades, atributos, relaciones o jerarquías ISA que

forman parte de un modelo interno. Ese modelo se utiliza posteriormente para validar el diagrama, transformarlo al modelo relacional y generar SQL.

Excalidraw

Excalidraw es una herramienta web de dibujo y diagramación con una estética similar a un boceto hecho a mano. Permite crear esquemas de forma rápida y colaborativa, por lo que resulta útil para representar ideas iniciales, diagramas informales o explicaciones visuales.

Su principal interés está en la facilidad de uso y en la libertad para dibujar cualquier tipo de esquema. No obstante, al igual que otras herramientas generales de diagramación, no está orientada específicamente a interpretar los elementos de un diagrama Entidad-Relación ni a aplicar reglas de validación propias de este modelo.

Por tanto, Excalidraw puede ser útil para realizar bocetos o explicar un diseño de manera informal, pero no cubre el objetivo principal de este proyecto: mantener una relación entre el diagrama visual, el modelo interno, la validación y la generación de SQL.

6.2. Herramientas orientadas al modelado de bases de datos

ERDPlus

ERDPlus es una herramienta web orientada al ámbito académico que permite trabajar con diagramas Entidad-Relación, esquemas relacionales y generación de SQL. Por este motivo, es una de las alternativas más cercanas al enfoque de este proyecto.

La diferencia principal es que este Trabajo Fin de Grado no consiste en crear una herramienta desde cero con todas esas funcionalidades, sino en evolucionar una aplicación heredada concreta. El trabajo se ha centrado en comprender el sistema existente, ampliar el soporte de elementos del modelo E/R, mejorar la validación, mantener la persistencia y ajustar la generación de SQL.

Además, el proyecto desarrollado mantiene una notación visual concreta y un alcance controlado, incorporando elementos como entidades débiles, atributos compuestos y multivaluados, relaciones ternarias y un soporte inicial para jerarquías ISA.

dbdiagram.io

dbdiagram.io representa un enfoque diferente. En lugar de centrarse principalmente en la construcción visual mediante arrastrar y configurar elementos, permite definir estructuras de base de datos mediante una sintaxis textual. A partir de esa descripción se genera el diagrama correspondiente.

Este enfoque puede resultar muy útil para usuarios con un perfil más técnico o acostumbrados a trabajar directamente con estructuras de tablas. Sin embargo, se aleja del objetivo principal de este proyecto, que consiste en trabajar desde una representación conceptual visual del modelo Entidad-Relación y transformarla después hacia el modelo relacional.

Frente a este tipo de herramientas, el editor desarrollado da más importancia al proceso de modelado visual y a la relación entre los elementos gráficos del diagrama y su significado dentro del modelo interno.

6.3. Comparativa frente a alternativas

La comparación entre herramientas debe entenderse teniendo en cuenta el objetivo concreto de este trabajo. Draw.io y Excalidraw son herramientas flexibles para dibujar diagramas, pero no están centradas en validar un modelo Entidad-Relación ni en generar SQL a partir de la semántica del diagrama. ERDPlus se aproxima más al enfoque académico de modelado de bases de datos, mientras que dbdiagram.io se orienta a la definición textual de estructuras relacionales.

El proyecto desarrollado no pretende competir con herramientas generales de diagramación ni con plataformas profesionales de diseño de bases de datos. Su objetivo es evolucionar un editor web académico de diagramas Entidad-Relación, manteniendo una correspondencia entre la representación visual, el modelo interno, la validación, la transformación relacional y la generación de SQL.

La Tabla 6.1 resume las diferencias principales entre algunas herramientas relacionadas y la aplicación desarrollada.

Tabla 6.1: Comparativa general entre herramientas relacionadas y la aplicación desarrollada.

Herramienta	Enfoque principal	Relación con bases de datos	Diferencia respecto a la aplicación desarrollada
diagrams.net / draw.io	Diagramación general	Permite dibujar diagramas E/R dentro de un conjunto amplio de diagramas.	Es una herramienta más generalista; no se centra en validar semánticamente un modelo E/R académico ni en transformarlo de forma guiada a SQL.
Excalidraw	Pizarra colaborativa y dibujo libre	Puede utilizarse para bocetar modelos, pero no está orientada específicamente a bases de datos.	Prioriza la libertad de dibujo frente a una estructura interna validable del modelo.
ERDPlus	Modelado de bases de datos	Permite crear diagramas E/R, esquemas relacionales y generar SQL.	Es una herramienta más especializada; la aplicación desarrollada se centra en evolucionar un editor académico heredado e incorporar soporte controlado para los elementos tratados en el proyecto.
dbdiagram.io	Diseño mediante texto	Usa DBML para definir estructuras de base de datos y generar diagramas.	Parte de una sintaxis textual, mientras que la aplicación desarrollada prioriza la edición visual directa del diagrama E/R.
Aplicación desarrollada	Editor académico E/R web	Representa diagramas E/R, los valida y genera SQL a partir del modelo interno.	Su interés está en la coherencia entre editor visual, modelo interno, validación, transformación relacional y alcance académico definido.

7. Conclusiones y líneas de trabajo futuras

En este capítulo se recogen las principales conclusiones obtenidas tras el desarrollo del proyecto y se señalan algunas posibles líneas de continuación. El objetivo no es repetir todo el trabajo realizado, sino valorar el resultado alcanzado, las decisiones tomadas durante el desarrollo y los aspectos que podrían ampliarse en el futuro.

7.1. Conclusiones

El desarrollo de este trabajo ha permitido evolucionar una aplicación web ya existente para el diseño de diagramas Entidad-Relación, ampliando sus capacidades y mejorando su coherencia interna. El proyecto no partía de cero, por lo que una parte importante del trabajo consistió en comprender el funcionamiento del sistema heredado, estabilizar algunos puntos delicados y adaptar su estructura para incorporar nuevas funcionalidades de forma controlada.

Una de las conclusiones principales es que, en una herramienta visual de modelado, no basta con representar correctamente los elementos en pantalla. Cada elemento del diagrama debe estar reflejado también en el modelo interno de la aplicación, ya que esa información se utiliza después para guardar el diagrama, reconstruirlo, validarlo, transformarlo al modelo relacional y generar el script SQL. Esta relación entre interfaz, modelo interno y generación de resultados ha sido uno de los aspectos más importantes del desarrollo.

A lo largo del proyecto se ha ampliado el soporte del modelo Entidad-Relación incorporando entidades débiles, atributos compuestos y multivaluados, relaciones ternarias con roles y un soporte inicial para jerarquías ISA. Estas ampliaciones no se limitaron a la parte visual, sino que también afectaron a la persistencia, la validación y la transformación relacional.

El soporte de jerarquías ISA se planteó de forma limitada y controlada. La aplicación permite representar una generalización con una o varias especializaciones, y transforma esta estructura generando una tabla para la entidad general y una tabla para cada especialización, donde la clave heredada actúa como clave primaria y clave foránea. No se incorporaron variantes más avanzadas como totalidad o parcialidad, especializaciones solapadas o exclusivas, discriminantes o herencia múltiple real. Esta decisión permitió implementar una primera base funcional sin aumentar el alcance de forma excesiva.

También se revisó la generación de SQL, especialmente el tratamiento de claves foráneas y la forma de expresar el script final. El objetivo fue obtener una salida más clara y coherente con el modelo relacional generado, reduciendo el uso de sentencias `ALTER TABLE` cuando las dependencias entre tablas pueden resolverse mediante el orden de creación.

Otro resultado relevante del proyecto es la mejora de la usabilidad del editor. Se reorganizó la interfaz, se revisaron los mensajes de validación y confirmación, se incorporaron ayudas dentro de la aplicación y se añadieron operaciones de apoyo como selección múltiple, ajuste de vista, exportación visual, importación JSON y generación de estructuras de ejemplo. Estas mejoras no cambian la notación Entidad-Relación, pero facilitan el uso de la herramienta y la revisión de los diagramas creados.

Durante el desarrollo también se ha comprobado la importancia de trabajar de forma incremental. Al tratarse de una aplicación heredada y visual, muchos cambios podían afectar a varias partes del sistema al mismo tiempo. Por ello, dividir el trabajo en tareas acotadas y combinar pruebas automáticas con revisión manual permitió avanzar con más seguridad y reducir el riesgo de introducir regresiones.

Al estar orientada a un contexto académico, la herramienta no busca sustituir la explicación teórica del modelo Entidad-Relación ni competir con soluciones profesionales de diseño de bases de datos. Su finalidad es servir como apoyo práctico para construir diagramas, validarlos y observar una posible traducción al modelo relacional y a SQL.

En conjunto, el resultado obtenido es una herramienta más completa que la aplicación inicial, con mayor soporte para elementos del modelo Entidad-Relación, una salida SQL más cuidada y una interfaz más clara.

7.2. Líneas de trabajo futuras

A pesar de las mejoras realizadas, el proyecto sigue ofreciendo varias posibilidades de ampliación. Algunas de ellas quedaron fuera del alcance de este trabajo por decisión de diseño, mientras que otras surgieron durante el desarrollo como mejoras posibles para versiones posteriores. Entre las líneas de trabajo futuras más relevantes se encuentran las siguientes:

- **Ampliar el soporte de jerarquías ISA.** La implementación actual cubre una versión inicial centrada en la herencia de clave desde la generalización hacia las especializaciones. Como trabajo futuro se podrían incorporar restricciones de totalidad o parcialidad, especializaciones solapadas o exclusivas y discriminantes. También podría estudiarse con más detalle si tendría sentido incorporar herencia múltiple real. Estas ampliaciones requerirían revisar la representación visual, el modelo interno, las validaciones y la transformación relacional.
- **Incorporar agregaciones.** Las agregaciones forman parte del modelo Entidad-Relación avanzado, pero no se implementaron en este trabajo. Su incorporación supondría representar relaciones tratadas como entidades de nivel superior, lo que afectaría al lienzo, a la persistencia, a la validación y a la generación del modelo relacional.
- **Estudiar una fusión semántica avanzada de diagramas.** La aplicación permite combinar un diagrama importado o una estructura generada con el contenido actual mediante una solución controlada basada en remapeo de identificadores, renombrado de conflictos y desplazamiento visual. Como trabajo futuro podría estudiarse una fusión más inteligente, capaz de detectar elementos equivalentes, reutilizar entidades ya existentes o resolver conflictos de forma guiada por el usuario. Esta ampliación tendría más complejidad, porque afectaría a nombres, referencias internas, relaciones, ISA, validación y persistencia.
- **Localizar nombres por defecto de nuevos elementos.** Actualmente la interfaz puede mostrarse en español o en inglés, pero los nombres creados por defecto para elementos del diagrama se mantienen como datos del usuario y no se traducen automáticamente. Una

posible mejora sería adaptar los nombres iniciales de nuevos elementos al idioma activo, sin modificar los elementos ya existentes al cambiar de idioma.

- **Permitir una configuración más detallada del SQL generado.** Actualmente la herramienta genera una estructura SQL suficiente para representar el esquema obtenido, pero no permite editar en detalle los tipos de datos, tamaños, restricciones adicionales o decisiones específicas de un sistema gestor concreto. Esta sería una posible ampliación si se quisiera acercar más la herramienta a un uso práctico avanzado.
- **Añadir funcionalidades avanzadas de edición.** La aplicación incorpora operaciones básicas sobre selección múltiple, como mover o borrar varios elementos seleccionados. En el futuro podrían incorporarse acciones más avanzadas como copiar y pegar fragmentos del diagrama, alinear o distribuir elementos, organizar automáticamente el contenido del lienzo o trabajar con varias vistas del mismo modelo. Estas mejoras aumentarían la comodidad de uso, aunque también introducirían nuevos retos relacionados con la sincronización entre el modelo interno y la representación visual.
- **Explorar persistencia remota y usuarios.** La aplicación se centra actualmente en el uso local, con persistencia en el navegador e importación y exportación mediante archivos JSON. Una posible evolución sería añadir un backend para guardar diagramas asociados a usuarios o compartir modelos, aunque esto supondría ampliar de forma importante el alcance técnico del proyecto.

En definitiva, el proyecto deja una base funcional sobre la que pueden construirse nuevas ampliaciones. Estas mejoras deberían abordarse de forma gradual, manteniendo el mismo criterio seguido durante este trabajo: introducir cambios acotados, comprobar su impacto en el modelo interno y asegurar que cada nueva funcionalidad se integre de forma coherente con la validación, la persistencia y la generación de SQL.

Bibliografía

- [1] Biome. Biome. <https://biomejs.dev/>. [Internet; consultado 25-mayo-2026].
- [2] Peter P. Chen. The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9–36, 1976.
- [3] ERDPlus. Erdplus. <https://erdplus.com/>. [Internet; consultado 15-junio-2026].
- [4] Excalidraw. Excalidraw. <https://excalidraw.com/>. [Internet; consultado 15-junio-2026].
- [5] Git. Git Documentation. <https://git-scm.com/docs>. [Internet; consultado 25-mayo-2026].
- [6] GitHub. GitHub. <https://github.com/>. [Internet; consultado 25-mayo-2026].
- [7] Holistics Software. dbdiagram.io Documentation. <https://docs.dbdiagram.io/>. [Internet; consultado 15-junio-2026].
- [8] JGraph. mxgraph. <https://github.com/jgraph/mxgraph>. [Internet; consultado 25-mayo-2026].
- [9] JGraph Ltd. diagrams.net / draw.io. <https://app.diagrams.net/>. [Internet; consultado 15-junio-2026].
- [10] Evil Martians. Lefthook. <https://lefthook.dev/>. [Internet; consultado 25-mayo-2026].

- [11] Jesús Maudes Raedo. Tema 6. El modelo E/R y su representación lógica en SQL. Material docente de Bases de Datos, Grado en Ingeniería Informática, Universidad de Burgos, 2017. Material docente consultado durante la realización del trabajo.
- [12] Meta. React. <https://react.dev/>. [Internet; consultado 25-mayo-2026].
- [13] Microsoft. Playwright. <https://playwright.dev/>. [Internet; consultado 25-mayo-2026].
- [14] Microsoft. Visual studio code. <https://code.visualstudio.com/>. [Internet; consultado 25-mayo-2026].
- [15] MUI. Material UI. <https://mui.com/material-ui/>. [Internet; consultado 25-mayo-2026].
- [16] npm. npm. <https://www.npmjs.com/>. [Internet; consultado 25-mayo-2026].
- [17] OpenJS Foundation. Node.js. <https://nodejs.org/>. [Internet; consultado 25-mayo-2026].
- [18] Overleaf. Overleaf. <https://www.overleaf.com/>. [Internet; consultado 25-mayo-2026].
- [19] Vercel. Vercel. <https://vercel.com/>. Accedido: 25 de junio de 2026.
- [20] Vitest. Vitest. <https://vitest.dev/>. [Internet; consultado 25-mayo-2026].